Práctico 01 — Shaders y kickoff del proyecto

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

21 de agosto de 2026



Lo que quedó resuelto el viernes pasado

Del array de C++ a la GPU — cerrado

El pipeline y qué etapas son fijas · dos máquinas, dos memorias · el VBO como almacenamiento de bytes sin tipo y el VAO con las instrucciones de cómo leerlos · `glDrawArrays`

Lo que quedó pendiente

En la actividad A había dos pasos que hablaban de **compilar** y **linkear** cada shader que todavía no se había explicado. Ordenarlos fue adivinar.

Hoy presentamos formalmente los **shaders**.

Proyecto integrador

Simulador de Acrobacias Aéreas

"Entorno 3D interactivo para representar una aeronave que pueda ejecutar un **circuito de acrobacias aéreas**, en donde el comportamiento de la aeronave se determina mediante un **modelo de dinámica de vuelo** que se provee como módulo".

- Aplicar los conocimientos de la materia en el desarrollo de una aplicación que genera imágenes sintéticas en **tiempo real** materia, **es** la materia.
- Cada viernes se dan los fundamentos para construir un módulo (o funcionalidad) del proyecto.
- Se acredita en el **examen final**, con entrega y defensa oral.

Ocho requerimientos mínimos

1. **Escenario 3D** — terreno con variaciones de altura y cielo.
2. **Modelo de aeronave** — con su actitud de vuelo correcta (pitch, roll, yaw).
3. **HUD** — al menos 3 instrumentos operativos
4. **Control de la aeronave** — FDM integrado con teclado y/o joystick.
5. **Sistema de cámaras** — al menos 3 vistas.
6. **Circuito** — mínimo 5 checkpoints, atravesados en orden y visualmente distinguibles.
7. **Detección de maniobras** — al menos 2 (looping, tonel).
8. **Sistema de puntuación** — por checkpoints y maniobras, visible en pantalla.

Al menos 2 mejoras extras (ej. niebla, nubes, ciclo día/noche, partículas, menú, colisiones, minimapa, alguna otra que quieran explorar) que mejoran la calificación, no la habilitan.

Lineamientos

- **Grupos de 2 estudiantes.** Para fomentar el trabajo en equipo.
- **OpenGL moderno (core profile), GLFW y GLM.** Otras librerías quedan a criterio del grupo, **con un límite**:
  - Los elementos gráficos principales (asociados a los contenidos de la materia) son de elaboración propia, no se delegan a un motor gráfico.
  - No se pueden usar repositorios de código fuente con proyectos similares y de proyectos desarrollados en la materia en años anteriores.
- Al integrar el FDM **no es necesario tocar su lógica física.**
  - Se conectan controles de entrada y se leen salidas con los datos de vuelo.
  - Se aceptan mejoras al FDM ☺
- **Se entrega** código fuente y un documento breve indicando: instrucciones de compilación, cómo se ejecuta y cuáles fueron las decisiones técnicas relevantes.

Cómo se acredita

Área	Peso
Presentación oral y defensa teórica	30 %
Coherencia técnica y funcionalidad	30 %
Calidad gráfica y mejoras sobre los mínimos	15 %
Interfaz y experiencia de usuario	15 %
Integración y calidad del código	10 %

Mínimo para aprobar: 60 % combinado

Importante!

Se asignarán los porcentajes **dando más importancia** a que pueden explicar **cómo** y **porqué** funciona el programa que a la prueba del programa funcionando (se supone que tiene que ejecutarse sin errores!).

Durante la defensa el tribunal puede pedir una modificación menor del código **en vivo**, o que se explique un fragmento cualquiera del mismo.

Herramientas de IA: permitidas, declaradas, defendibles

1. **Permitidas.** No están prohibidas y no hay penalización por usarlas.
2. **Declaradas.** La entrega indica **qué partes** contaron con asistencia, junto con la declaración de autoría de los dos integrantes.
3. **Defendibles.** Cada integrante tiene que poder explicar y defender **cualquier** fragmento del código entregado — lo haya escrito, lo haya pedido o lo haya copiado de un repositorio.

Lo que se evalúa no es quién tipeó:
es **si se puede explicar, romper y arreglar**.

Roadmap de módulos/funcionalidades

Viernes	Lo que se construye	Req.
hoy	shaders: la pieza que dibuja	—
28-ago	primitivas 3D: el cubo, `Mesh` y `Shader`	2
04-sep	cono y cilindro; matriz de modelo	2
11-sep	la aeronave armada y orientada	2
18-sep	cámara en la escena	5
25-sep	FDM, game loop e input: **el avión vuela**	4
octubre	terreno, cielo, HUD, profundidad, texturas	1, 3
noviembre	circuito, maniobras y puntuación	6, 7, 8

Las fechas de octubre y noviembre se publican cerradas después del primer parcial.

¿Qué es un "shader"?

Un programa que corre **una vez por vértice**
o **una vez por fragmento**.

- En paralelo, muchas invocaciones a la vez.
- Cada una **no sabe nada de las demás**: no ve el vértice de otra invocación, ni cuántos hay, ni en qué orden salieron.
- Por eso adentro de un shader no hay un bucle que recorre "todos los vértices", ni una variable que se acumule entre invocaciones.
- Se escribe el comportamiento deseado para **una** invocación. La GPU lo ejecuta tantas veces como haga falta.

¿En qué se escribe?

GLSL — *OpenGL Shading Language*

Parte del estándar de Khronos, en el core desde 2004 (OpenGL 2.0).

- Sintaxis **parecida a C**: tipos, funciones, `if`, `for`, un `main()`.
- Es un lenguaje orientado a operaciones vectoriales y geométricas, por lo que dispone:
  - vectores y matrices como tipos nativos — `vec3`, `vec4`, `mat4`;
  - acceso por componente — `v.x`, `v.xyz`, `v.rgb`;
  - funciones especiales para operaciones vectoriales — `dot`, `normalize`, `mix`.
- Lo que **no tiene**: punteros, `new` / `malloc`, recursión, ni la librería estándar de C. Adentro de un shader no disponemos de `printf`.

→ No es C, y no se compila con el compilador de C++.

Lenguajes en otras APIs

API	Lenguaje de shaders	
OpenGL	**GLSL**	es lo que vamos a usar
Direct3D (Microsoft)	HLSL	*High Level Shading Language*
Metal (Apple)	MSL	basado en C++
Vulkan	**SPIR-V**	formato binario; GLSL y HLSL se traducen a él

Distintos nombres para lo mismo

Lo que en *OpenGL* es **fragment shader**, en *Direct3D* se llama **pixel shader**.

El nombre de OpenGL es el más genuino: un fragmento es un *candidato* a píxel — todavía puede descartarse antes de llegar al framebuffer.

> Las etapas del pipeline son las mismas. Cambian el nombre y la sintaxis, no el modelo.

Firma mínima

vertex shader

```
#version 460 core

layout (location = 0) in vec3 aPos;

void main()
{
    gl_Position = vec4(aPos, 1.0);
}
```

fragment shader

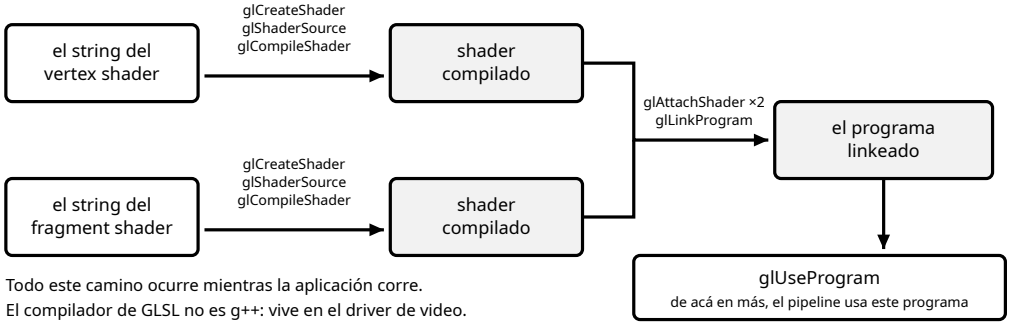
```
#version 460 core

out vec4 FragColor;

void main()
{
    FragColor = vec4(1.0, 0.5, 0.2, 1.0);
}
```

- `#version 460 core` — primera línea, versión de la API a usar (define qué funcionalidades están disponibles).
- Listado de variables globales (algunas pueden pertenecer al pipeline de ejecución)
- Calificadores `out` e `in` permiten pasar / recibir información en el pipeline de ejecución: `out` lo que sale, `in` lo que entra
- `layout (location = 0)` — **con cuál** de los atributos declarados del lado de C++ se conecta la variable. El número tiene que coincidir.
- `main()` sin argumentos
- El vertex shader **está obligado** a escribir `gl_Position`. Si no lo hace, no hay nada que rasterizar.

De string a programa



Todo este camino ocurre mientras la aplicación corre.
El compilador de GLSL no es g++: vive en el driver de video.

Nota: No es el único camino: desde OpenGL 4.6 se le puede entregar el shader **ya traducido** a un formato binario intermedio (**SPIR-V**), generado fuera de línea al compilar el proyecto.

Proceso de compilación

- En la primera pasada de compilación (compilador de C++) no hay errores. Los shaders están como strings, y los strings siempre compilan.
- El compilador de GLSL vive en el driver de video y corre cuando la aplicación ya se está ejecutando.
- Si el shader no compila, no hay error de g++, no hay warning, no hay nada.
- **Hay que pedir el resultado de la compilación del shader.**

```
GLint ok = 0;
glGetShaderiv(vs, GL_COMPILE_STATUS, &ok);
if (!ok) {
    GLint largo = 0;
    glGetShaderiv(vs, GL_INFO_LOG_LENGTH, &largo);  // incluye terminador '\0'
    std::string log(largo, '\0');
    glGetShaderInfoLog(vs, largo, nullptr, log.data());
    std::cout << "Vertex shader: " << log << std::endl;
}
```

El tamaño del log **se consulta**. En muchos tutoriales usan un largo fijo `char log[512]` que trunca el mensaje justo cuando es largo, que es cuando más se lo necesita.

GLSL nivel inicial (lo que vimos hasta ahora)

```
#version 460 core      // siempre, versión de la API a usar

in  vec3 aPos;         // entra (por vértice)
out vec4 FragColor;    // sale

layout (location = 0) // con qué atributo
                      // se conecta

void main() { ... }   // el cuerpo de UNA
                      // invocación

gl_Position           // obligatorio en el
                      // vertex shader

vec4(aPos, 1.0)       // constructor: arma un
                      // vec4 con un vec3 + 1
```

Otras funcionalidades

- `uniform` — 28-ago
- matrices — 02-sep
- texturas y `sampler2D` — oct
- normales e iluminación — oct/nov

Especificación completa del lenguaje (Khronos) (también disponible en el aula virtual):
https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf

Checklist de pantalla negra

Antes de correr el programa: **pensar qué se espera ver**.

1. ¿**Compiló** el shader?
   `glGetShaderiv(..., GL_COMPILE_STATUS, ...)` + `glGetShaderInfoLog`
2. ¿**Linkeó** el programa?
   `glGetProgramiv(..., GL_LINK_STATUS, ...)` + `glGetProgramInfoLog`
3. ¿Está **activo el VAO** (bind) en el momento de dibujar?
4. ¿Los **vértices** están en el rango $[-1; 1]$?
5. ¿**Winding / culling**?
   hoy `GL_CULL_FACE` está deshabilitado: es el último sospechoso

Las últimas tres verificaciones las tenemos que hacer "manualmente"

Romperlo a propósito

- Cuando el triángulo aparezca, **no dejarlo ahí**: sacarle un punto y coma al shader y volver a correr.
  - ¿Salió el mensaje del log? Entonces el instrumento funciona.
  - ¿No salió nada? Entonces nunca se iba a avisar — y la próxima pantalla negra cuesta una tarde.

Un chequeo que nunca falló todavía no es un chequeo.

Clínica: romper el triángulo, en este orden

	Qué se saca o se cambia
1	La línea que **habilita el atributo** del VAO
2	Un punto y coma del **vertex shader** — y no mirar el log
3	El vértice de arriba: 0,5 → 1,5

La regla del bloque

Antes de correr, escribir en el papel qué se espera ver. Una predicción equivocada es el único resultado que enseña algo.

Al terminar cada una, dejar el código andando otra vez.

Qué mostró cada rotura

1. **Atributo apagado** → pantalla negra sin ningún error. Los tres vértices reciben $(0, 0, 0, 1)$ y el triángulo se degenera en un punto.

   La checklist no puede detectarla: se determina "manualmente" sobre el código.

2. **Shader roto, log no consultado** → pantalla negra, sin mensaje, y el programa **compiló perfecto**.

   El shader es un string hasta que la aplicación arranca: el compilador vive en el driver.

3. **Vértice en** $1{,}5$ → el triángulo no desaparece: aparece **recortado** por el borde superior.

   Hoy los vértices se escriben a mano en $[-1; 1]$.

Tres pantallas negras distintas. **Ninguna se diagnostica tanteando.**

Actividades para el viernes que viene

1. **El triángulo corriendo en su máquina.**
2. Versión **mínima operativa de uno de los módulos** para el proyecto:
   - `ResourceManager`: C++ puro, sin OpenGL.
   - Funcionalidad: leer los fuentes de un shader desde archivo y mantener copia disponible para otros módulos. (ver enunciado)
   - La interfaz del enunciado es una **sugerencia**
   - Vayan registrando las decisiones para la presentación final.

El enunciado que sirve de guía está disponible en el aula virtual.

Dónde estamos

- **Hoy:** cierre del primer plasmado propio — datos en la GPU y dos los shaders mínimos necesarios. Es la base donde se construye todo lo que sigue.
- **28-ago:** los shaders salen a archivos `.vs` / `.fs` — con el ResourceManager que tienen de tarea — y agregaremos algunas clases más al proyecto.
- **04-sep:** matrices. Los vértices dejan de escribirse a mano en $[-1; 1]$.

¿Cuántos vértices tiene un cubo?